

Conception et développement d'IHM

Introduction et prérequis

Introduction et prérequis: contenu

- Introduction à la conception et développement d'Interfaces Homme-Machine (IHM)
- Développement natif vs multiplateforme
- Présentation de Flutter
- Présentation de React
- Présentation de React Native
- Flutter vs React Native
- JavaScript ES6
- L'écosystème React

Introduction à la conception et développement d'Interfaces Homme-Machine (IHM)

Introduction à la conception et développement d'IHM

IHM = ensemble des dispositifs matériels et logiciels permettant à un utilisateur d'interagir avec un système interactif.



- IHM point clé d'une application
- Souvent facteur d'échec ou de réussite
 - Bon logiciel + mauvaise IHM = échec
 - Logiciel correct + très bonne IHM = réussite
- Source d'erreurs, de coûts supplémentaires
 - IHM = 50% des coûts de développement + 50% des demandes de modification

La conception et développement d'IHM dans les années 90



La conception et développement d'IHM en 2024



Développement natif vs multiplateforme

La différence fondamentale entre le développement natif et le développement multiplateforme tient au système d'exploitation pour lequel vous développez. Le développement mobile natif vous permet de créer des applications **pour un système d'exploitation particulier** - Android ou iOS. Le développement mobile multiplateforme, en revanche, vous permet de créer des applications destinées à **plusieurs systèmes d'exploitation**.

C'est quoi le développement natif ?

Les applications mobiles/desktop... natives sont des applications développées pour Android, iOS ou desktop. Selon le système d'exploitation pour lequel vous développez, vos applications sont généralement codées dans un langage de programmation spécifique. Les applications Android natives sont écrites à l'aide des langages de programmation Java ou Kotlin. Si vous développez des applications iOS natives, vous utilisez les langages de programmation Objective-C ou Swift.

Outre les kits de développement logiciel (SDK) spécifiques au système d'exploitation, le développement mobile natif nécessite un environnement de développement intégré (IDE). Pour les applications Android, vous devez utiliser Android Studio ou IntelliJ IDEA. Ces outils fonctionnent sous Windows, macOS ou Linux. Pour les applications iOS, vous devez utiliser Xcode ou AppCode comme IDE. Ces outils ne fonctionnent que sur macOS.

Avantages et inconvénients du développement natif

Avantages

- **Meilleures performances** : les applications natives sont plus rapides, plus efficaces et plus réactives aux actions des utilisateurs.
- **Sécurité renforcée** : Les applications natives peuvent renforcer la sécurité des données.
- **Qualité de l'interface utilisateur** : les applications natives offrent à l'utilisateur une expérience d'entrée et de sortie plus fluide. L'aspect et la convivialité sont cohérents car elles héritent des interfaces du système d'exploitation de l'appareil.
- **Accès à l'ensemble des fonctionnalités** : Les applications mobiles natives ont accès à toutes les caractéristiques et fonctionnalités de l'appareil.
- **Un minimum de bogues** : Les développeurs d'applications natives ont accès aux mises à jour des kits de développement logiciel (SDK) dès qu'elles sont publiées.

Inconvénients

- **Coût** : Les applications mobiles natives sont généralement coûteuses à déployer. Vous devez disposer d'équipes de développeurs distinctes pour les plates-formes spécifiques.
- **Temps de développement** : Des équipes séparées travaillant sur des applications similaires pour des plateformes distinctes prennent du temps. La maintenance et les améliorations nécessitent également beaucoup d'efforts.
- **Manque de réutilisation du code** : Vous devez créer et conserver le code dans des projets distincts pour cibler différents systèmes d'exploitation mobiles.

Développement natif vs multiplateforme

Développement multiplateforme:

Vous créez des applications mobiles **multiplateformes** à partir d'une **base de code unique**. L'objectif du développement d'applications multiplateformes est de **cibler différents systèmes d'exploitation avec un seul projet**. Vous créez ces applications à l'aide de Frameworks multiplateformes, qui utilisent des SDK spécifiques à chaque plateforme (SDK Android et SDK iOS) à **partir d'une API unifiée**. Cela vous permet d'accéder facilement aux SDK et aux bibliothèques des différentes plateformes. Ces Frameworks sont créés par des entreprises privées.

Les deux Frameworks multiplateformes les plus populaires sont:

- **React Native** de Meta. Il utilise JavaScript comme langage de programmation.
- **Flutter** de Google. Il utilise Dart comme langage de programmation.

Les applications mobiles multiplateformes sont généralement compilées pour utiliser des éléments d'interface utilisateur natifs qui donnent à l'application un aspect natif. Comme indiqué précédemment, elles fournissent une abstraction aux SDK des plateformes sous-jacentes. Les capteurs exposés comprennent l'accès au GPS, au niveau de batterie, à l'appareil photo et au microphone. Parmi les exemples bien connus d'applications mobiles multiplateformes, on peut citer :

- Facebook, Skype, Walmart et Airbnb (React Native)
- Google Ads, My BMW App, eBay Motors et le New York Times (Flutter)

Avantages et inconvénients du développement multiplateforme

Avantages

- **Faibles coûts** : Le développement mobile multiplateforme ne nécessite qu'une seule équipe de développeurs.
- **Réutilisation du code** : Les Frameworks multiplateforme vous permettent d'utiliser une base de code unique.
- **Développement rapide** : La réutilisation du code et l'amélioration de la productivité permettent une mise sur le marché plus rapide.
- **Maintenance plus facile** : La mise à jour d'une application multiplateforme est plus facile.

Inconvénients

- **Empreinte numérique plus importante** : Les applications multiplateformes sont généralement plus volumineuses en taille.
- **Intégration difficiles** : Certaines intégrations spécifiques au matériel, comme l'utilisation des GPU, peuvent nécessiter des compétences en matière de développement d'applications natives.
- **Performances moindres** : Les Frameworks multiplateformes intègrent généralement un moteur d'exécution personnalisé pour exécuter les applications. Cela ajoute une couche de calcul supplémentaire, ce qui peut entraîner une baisse des performances.
- **Fonctionnalités de la plateforme retardées** : Les nouvelles versions des SDK comprennent généralement de nouvelles fonctionnalités et des mises à jour. Avec les Frameworks multiplateformes, vous devez attendre une mise à jour séparée pour accéder à ces nouvelles fonctionnalités.

Présentation de Flutter

Flutter est un Framework open-source créé par Google pour le développement d'applications multiplateformes. Lancé en 2017, il offre aux développeurs un moyen efficace de créer des applications mobiles, web et de bureau à partir d'une seule base de code.

Caractéristiques principales :

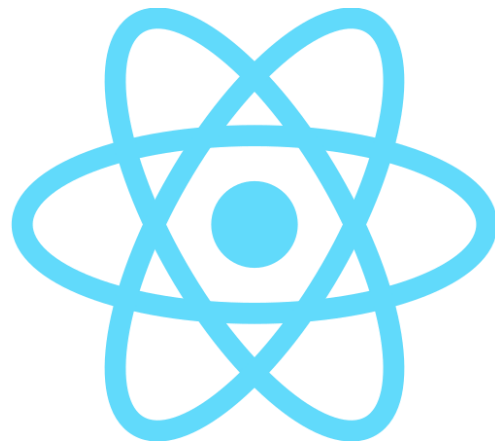
- **Widgets personnalisables** : Flutter offre une large gamme de widgets hautement personnalisables permettant aux développeurs de créer des interfaces utilisateur riches et dynamiques.
- **Performances élevées** : Grâce à son moteur de rendu intégré, Flutter garantit des performances fluides et rapides sur différentes plateformes, sans compromis sur la qualité visuelle.
- **Hot Reload** : L'une des fonctionnalités les plus appréciées de Flutter est le Hot Reload, qui permet aux développeurs de voir instantanément les changements apportés au code sans avoir à re-compiler l'application, accélérant ainsi le processus de développement.
- **Langage Dart** : Flutter utilise le langage de programmation Dart, également développé par Google, qui est facile à apprendre et offre des fonctionnalités modernes telles que le typage fort et la gestion efficace de la mémoire.
- **Support multiplateforme** : Avec Flutter, vous pouvez créer des applications pour Android, iOS, web et desktop à partir d'une seule base de code, ce qui réduit le temps et les efforts nécessaires pour développer et maintenir des applications sur différentes plateformes.



Présentation de React

React (également connu sous le nom de React.js ou ReactJS) est une bibliothèque JavaScript à code source ouvert permettant de créer des interfaces utilisateur ou des composants d'interface utilisateur. Elle est maintenue par Facebook et une communauté de développeurs individuels et d'entreprises.

React permet de construire des interfaces utilisateur composables. Elle encourage la création de composants d'interface utilisateur réutilisables, qui présentent des données qui évoluent dans le temps. Beaucoup de gens utilisent React comme le V de MVC (Modèle-Vue-Contrôleur). React fait abstraction du DOM, offrant un modèle de programmation plus simple et de meilleures performances. React peut également effectuer un rendu sur le serveur à l'aide de Node, et il peut alimenter des applications natives à l'aide de React Native. React met en œuvre un flux de données réactif unidirectionnel, ce qui réduit le nombre d'erreurs et facilite le raisonnement par rapport à la liaison de données traditionnelle.



Présentation de React Native

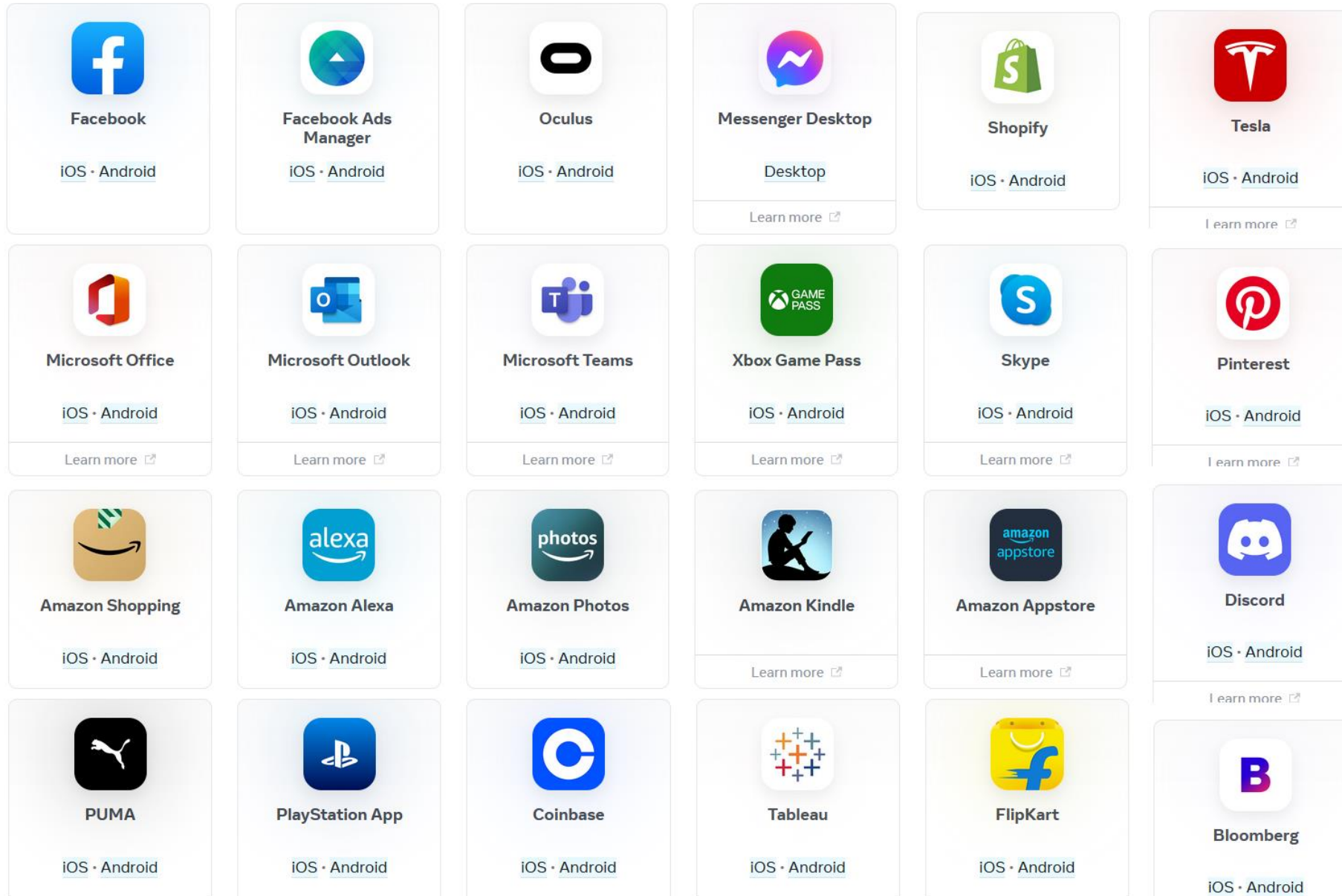
React Native (également connu sous le nom de RN) est un Framework d'application mobile populaire basé sur JavaScript qui permet de construire des applications mobiles avec rendu natif pour iOS et Android. Le Framework permet de créer une application pour différentes plateformes en utilisant la même base de code. React Native a été publié pour la première fois par Facebook en tant que projet open-source en 2015. En quelques années, il est devenu l'une des principales solutions utilisées pour le développement mobile. React Native est utilisé pour créer certaines des principales applications mobiles au monde, notamment **Instagram**, **Facebook** et **Skype**. Plusieurs raisons expliquent le succès mondial de React Native. Tout d'abord, en utilisant React Native, les entreprises peuvent créer du code une seule fois et l'utiliser pour créer à la fois leurs apps iOS, Android, et desktop. Cela se traduit par d'énormes économies de temps et de ressources. Deuxièmement, React Native a été construit sur la base de React - une bibliothèque JavaScript, qui était déjà extrêmement populaire lorsque le Framework mobile a été lancé.



React Native: les plateformes supportées



React Native: exemples d'applications



Flutter vs React Native

	Flutter	React Native
Date de lancement	2017	2015
Composants UI	Widgets propriétaires	Composants OS natifs
Langage	Dart	JavaScript
Apparence native	★ ★ ★	★ ★ ★ ★ ★
Rapidité/réactivité	★ ★ ★ ★	★ ★ ★
Adoption par l'industrie	★	★ ★ ★ ★
Documentation	★ ★ ★ ★	★ ★ ★
Communauté	★ ★ ★	★ ★ ★ ★ ★
# Offres emploi Maroc 03/2024 (LinkedIn)	34	49 + (269 React)
# Offres emploi France 03/2024 (LinkedIn)	333	912 + (3321 React)
# Offres emploi Monde 03/2024 (LinkedIn)	8k	35k + (140k React)

JavaScript ES6

JavaScript ES6

ES6, abréviation de "ECMAScript 6", est une version majeure de la spécification ECMAScript, qui est le standard sur lequel JavaScript est basé. ES6 a été officiellement publié en juin 2015. Cette version a apporté de nombreuses fonctionnalités et améliorations au langage JavaScript, ce qui a considérablement enrichi sa syntaxe et sa capacité à traiter les problèmes de développement modernes. Certaines des fonctionnalités les plus notables introduites dans ES6 comprennent :

- ✓ Les fonctions fléchées : Une syntaxe plus concise pour définir des fonctions anonymes.
- ✓ La déclaration de variables avec `let` et `const` : Ces mots-clés permettent de déclarer des variables avec un contexte de portée local.
- ✓ Les templates de chaînes : Permettent de concaténer des chaînes de caractères de manière plus lisible.
- ✓ Les paramètres par défaut : Permettent de définir des valeurs par défaut pour les paramètres de fonction.
- ✓ Les destructurations : Permettent d'extraire des valeurs d'objets ou de tableaux dans des variables distinctes.
- ✓ Les classes : Une syntaxe plus claire pour la définition de classes et l'héritage.
- ✓ Les modules : Permettent d'organiser le code en modules réutilisables et indépendants.
- ✓ Les Promesses : Un mécanisme pour gérer les opérations asynchrones de manière plus claire et plus efficace.

La maîtrise d'ES6 est un prérequis essentiel pour le développement React et React Native

JavaScript ES6: les fonctions fléchées

Il existe deux manières différentes d'écrire une fonction en JavaScript : la manière traditionnelle, en utilisant le mot-clé `function`, appelée déclaration de fonction, et en tant que fonction fléchée, qui a été introduite dans ES6.

Le principal avantage des fonctions fléchées est leur caractère succinct. Nous pouvons utiliser plusieurs abréviations pour écrire nos fonctions afin d'éliminer les éléments superflus, de sorte que nous pouvons même tout écrire sur une seule ligne.

```
function MyComponent(props) {  
    return props.content;  
}  
  
// Syntaxe de fonction fléchée  
const MyComponent = (props) => {  
    return props.content;  
};  
  
// On peut enlever « return » et les accolades pour les fonctions d'une seule ligne  
const MyComponent = (props) => props.content;  
  
// On peut aussi enlever les parenthèses pour les fonctions ayant un seul argument  
const MyComponent = props => props.content;
```

JavaScript ES6: les fonctions fléchées

Les fonctions fléchées sont également très utiles dans les situations où il n'est pas nécessaire de nommer la fonction, comme dans le cas des fonctions de rappels « callback functions ».

```
const myArray = [1, 2, 3];  
  
// From this:  
myArray.forEach(function (item) {  
  console.log(item);  
});  
  
// To this:  
myArray.forEach(item => console.log(item));
```

JavaScript ES6: les fonctions fléchées

Une différence importante entre la syntaxe de la fonction arrow et celle de la fonction normale réside dans la manière dont elles traitent le mot-clé **this**. Dans une fonction normale, ce mot-clé fait référence à l'objet à partir duquel vous appelez la fonction. Dans une fonction fléchée, le mot-clé **this** fait référence à l'objet à partir duquel vous définissez la fonction. Supposons que vous ayez une personne avec les propriétés et les méthodes suivantes :

```
const person = {  
  name: 'ALAMI',  
  skills: ['HTML', 'CSS', 'JavaScript'],  
  
  showSkills() {  
    this.skills.forEach(function (skill) {  
      console.log(`${this.name} is skilled in ${skill}`);  
    });  
  },  
};  
  
person.showSkills();
```

Si vous exécutez le code ci-dessus, le résultat de l'appel à la méthode `showSkills()` sera comme suit :

- undefined is skilled in HTML
- undefined is skilled in CSS
- undefined is skilled in JavaScript

JavaScript ES6: les fonctions fléchées

Ici, le mot-clé **this** fait référence à l'objet global Window car nous avons appelé la méthode showSkills() en dehors de l'objet person. Dans l'objet global, la propriété name est indéfinie

```
const person = {  
  name: 'ALAMI',  
  skills: ['HTML', 'CSS', 'JavaScript'],  
  
  showSkills() {  
    this.skills.forEach(function (skill) {  
      console.log(`${this.name} is skilled in ${skill}`);  
    });  
  },  
};  
  
person.showSkills();
```

JavaScript ES6: les fonctions fléchées

Réécrivons maintenant la fonction de rappel en utilisant la syntaxe fléchée:

```
const person = {  
  name: 'ALAMI',  
  skills: ['HTML', 'CSS', 'JavaScript'],  
  
  showSkills() {  
    this.skills.forEach(skill => console.log(`${this.name} is skilled in ${skill}`) );  
  },  
};  
  
person.showSkills();
```

Si vous exécutez le code ci-dessus, le résultat de l'appel à la méthode showSkills() sera comme suit :

ALAMI is skilled in HTML

ALAMI is skilled in CSS

ALAMI is skilled in JavaScript

Ici, le mot-clé **this** fait référence à l'objet à partir duquel la fonction fléchée est définie, c'est-à-dire l'objet person. C'est ce comportement qui fait que les fonctions fléchées sont préférées, car il est plus logique que this fasse référence à l'objet à partir duquel vous définissez une fonction plutôt qu'à celui à partir duquel vous l'appellez.

JavaScript ES6: Template Literals

Avec ES6, nous avons reçu une nouvelle forme de chaîne de caractères appelée "template literal", qui consiste en deux ticks arrière `` (CTRL + ALT + 7) au lieu de guillemets simples ou doubles. Au lieu d'utiliser l'opérateur +, nous pouvons connecter des chaînes en plaçant une expression JavaScript (telle qu'une variable) dans une syntaxe spéciale \${} :

```
function sayHello(text) {  
    return "Hello " + text + "!";  
}  
  
// Avec Template Literals  
  
function sayHelloAgain(text) {  
    return `Hello again, ${text}!`;  
}
```

JavaScript ES6: Template Literals

Template literals facilitent également le travail avec les chaînes de caractères lorsqu'il s'agit de chaînes de caractères à plusieurs lignes. Pour les chaînes de caractères normales, vous devez utiliser une combinaison du signe plus + et de \n pour indiquer une nouvelle ligne.

```
const regularString =  
'Hello there! \n' +  
'Welcome to our website. \n' +  
'How can we help you today?'  
  
const templateLiteral =  
`Hello there!  
Welcome to our website.  
How can we help you today?`  
  
console.log(regularString)  
console.log(templateLiteral)
```

JavaScript ES6: Conditionnels courts : &&, ||, Opérateur ternaire

À l'aide de quelques opérateurs JavaScript essentiels, nous réduisons les répétitions et rendons notre code plus concis. Nous pouvons transformer l'instruction if ci-dessus en l'instruction suivante, en utilisant l'opérateur ternaire. L'opérateur ternaire fonctionne exactement de la même manière qu'une instruction if, mais il est plus court, c'est une expression (et non une instruction), et il peut être inséré dans JSX de React

```
function App() {  
  const isLoggedIn = true;  
  
  if (isLoggedIn) {  
    return 'Welcome back!';  
  }  
  
  return 'Who are you?';  
}
```

```
function App() {  
  const isLoggedIn = true;  
  
  return isLoggedIn ? 'Welcome back!' : 'Who are you?';  
}
```

JavaScript ES6: Conditionnels courts : &&, ||, Opérateur ternaire

Si nous devons modifier l'exemple ci-dessus et ne vouloir afficher le texte que si l'utilisateur est connecté (si `isLoggedIn` est vrai), ce serait un excellent cas d'utilisation de l'opérateur `&&` (et). Si la première valeur (opérande) du conditionnel est vraie, l'opérateur `&&` affiche le second opérande. Dans le cas contraire, il renvoie la première opérande. Et comme il est `false` (c'est-à-dire qu'il s'agit d'une valeur automatiquement convertie en booléen `false` par JavaScript), il n'est pas rendu par JSX.

```
import React from "react";

function App() {
  const isLoggedIn = true;

  return <div>{isLoggedIn && "Welcome back!"}</div>;
}

export default App;
```

JavaScript ES6: Conditionnels courts : &&, ||, Opérateur ternaire

Disons que nous voulons faire l'inverse de ce que nous faisons actuellement : ne dire "Who are you?" que si `isLoggedIn` est faux. Si c'est vrai, nous n'afficherons rien. Pour cette logique, nous pouvons utiliser l'opérateur `||` (ou). Il fonctionne essentiellement à l'inverse de l'opérateur `&&`. Si le premier opérande est vrai, le premier opérande (faux) est retourné. Si le premier opérande est faux, le second opérande est renvoyé.

```
import React from "react";

function App() {
  const isLoggedIn = true;

  return <div>{isLoggedIn || "Who are you?"}</div>;
}

export default App;
```

JavaScript ES6: Trois méthodes de tableau : .map(), .filter(), .reduce()

La fonction Array.map():

La méthode Array.map() permet d'itérer sur un tableau et de modifier ses éléments à l'aide d'une fonction de rappel. La fonction de rappel sera ensuite exécutée sur chacun des éléments du tableau.

Exemple:

On veut multiplier chacun des éléments d'un tableau par 3. On peut utiliser une boucle for comme suit :

```
let arr = [3, 4, 5, 6];

for (let i = 0; i < arr.length; i++){
    arr[i] = arr[i] * 3;
}

console.log(arr); // [9, 12, 15, 18]
```

Mais vous pouvez utiliser la méthode Array.map() pour obtenir le même résultat. Voici un exemple :

```
let arr = [3, 4, 5, 6];

let modifiedArr = arr.map(function(element){
    return element * 3;
});

console.log(modifiedArr); // [9, 12, 15, 18]
```

JavaScript ES6: Trois méthodes de tableau : .map(), .filter(), .reduce()

La fonction `Array.filter()`:

La méthode `Array.filter()` est une méthode ES6 qui fournit une syntaxe plus propre pour filtrer un tableau. Elle renvoie de nouveaux éléments dans un nouveau tableau sans modifier le tableau d'origine.

Syntaxe:

```
var newArray = array.filter(function(item) {  
    return condition;  
});
```

Exemple:

```
var numbers = [1, 3, 6, 8, 11];  
var greaterThanSeven = numbers.filter(function(number) {  
    return number > 7;  
});  
  
console.log(greaterThanSeven);
```

JavaScript ES6: Trois méthodes de tableau : .map(), .filter(), .reduce()

La fonction `Array.reduce()`:

La fonction `reduce()` est utilisée dans la manipulation de données qui réduit un tableau à une seule valeur. Elle prend deux paramètres : un accumulateur et l'élément courant d'un tableau. Elle permet notamment de calculer la somme ou le produit de tous les éléments, de trouver les valeurs maximale et minimale, d'aplatir un tableau, etc.

Syntaxe: `arr.reduce(callback(accumulator, currentValue), initialValue)`

Exemple:

```
const numbers = [1, 2, 3, 4, 5, 6];

function sum_reducer(accumulator, currentValue) {
  return accumulator + currentValue;
}

let sum = numbers.reduce(sum_reducer);
console.log(sum); // 21

// Avec la syntaxe fléchée
let summation = numbers.reduce(
  (accumulator, currentValue) => accumulator + currentValue
);

console.log(summation); // 21
```

JavaScript ES6: POO - Property Shorthand, Destructuring, Spread Operator

Property Shorthand:

Pour ajouter des propriétés à un objet lors de sa création, vous devez nommer la propriété et sa valeur correspondante. Un raccourci très simple à retenir est que si le nom de la propriété est le même que la valeur, il suffit d'énumérer le nom de la propriété. Il s'agit de l'abréviation de la propriété de l'objet

```
const name = "ALAMI";

const user = {
  // instead of name: name, we can use...
  name,
};

console.log(user.name); // ALAMI
```

JavaScript ES6: POO - Property Shorthand, Destructuring, Spread Operator

Destructuring:

La manière standard d'accéder aux propriétés d'un objet est d'utiliser la notation par points. Une approche encore plus pratique, cependant, est la déstructuration des objets. Elle nous permet d'extraire les propriétés d'un objet donné sous la forme de variables individuelles portant le même nom. C'est un peu comme si vous écriviez un objet à l'envers, ce qui rend le processus intuitif. C'est bien plus agréable à utiliser que de devoir utiliser le nom de l'objet plusieurs fois pour y accéder chaque fois que l'on veut en extraire une valeur

```
const user = {  
  name: "ALAMI",  
  age: 28,  
  isEnglishSpeaker: true,  
};  
  
const name = user.name;  
const age = user.age;  
  
// Object destructuring  
const { age, name, isEnglishSpeaker } = user;
```

JavaScript ES6: POO - Property Shorthand, Destructuring, Spread Operator

Spread Operator:

Si vous souhaitez créer des objets à partir d'objets existants, vous pouvez énumérer les propriétés une par une, mais cela peut s'avérer très répétitif. Au lieu de copier les propriétés manuellement, vous pouvez répartir toutes les propriétés d'un objet dans un autre objet (lorsque vous le créez) à l'aide de l'opérateur de répartition d'objet :

```
const user = {  
  name: "ALAMI",  
  age: 28,  
  isEnglishSpeaker: true,  
};  
  
const firstUser = {  
  name: user.name,  
  age: user.age,  
  isEnglishSpeaker: user.isEnglishSpeaker,  
};  
  
const secondUser = {  
  ...user,  
};
```

JavaScript ES6: POO - Property Shorthand, Destructuring, Spread Operator

L'avantage de l'extension d'objet est que vous pouvez intégrer autant d'objets que vous le souhaitez dans un nouvel objet, et vous pouvez les ordonner comme des propriétés. Mais attention, les propriétés portant le même nom qui seront créées ultérieurement écraseront les propriétés précédentes.

```
const user = {
  name: "ALAMI",
  age: 28,
};

const moreUserInfo = {
  age: 70,
  country: "USA",
};

// Copier toutes les propriétés de l'utilisateur dans secondUser
const secondUser = {
  ...user,
  ...moreUserInfo,
  computer: "MacBook Pro",
};

console.log(secondUser);
// { name: "ALAMI", age: 70, country: "USA", computer: "Macbook Pro" }
```

JavaScript ES6: Promises

Pratiquement toutes les applications React sont constituées de code asynchrone - du code qui prend un temps indéfini pour être exécuté. En particulier si vous avez besoin d'obtenir ou de modifier des données à partir d'une API externe en utilisant des fonctionnalités du navigateur telles que l'API Fetch ou la bibliothèque tierce axios.

Les promesses sont utilisées pour résoudre du code asynchrone afin qu'il soit résolu comme du code synchrone normal, que nous pouvons lire de haut en bas.

La syntaxe du constructeur d'un objet promesse est la suivante:

```
let promise = new Promise(function(resolve, reject) {  
  // executor  
});
```

La fonction passée à la nouvelle Promesse est appelée l'exécuteur. Lorsqu'une nouvelle promesse est créée, l'exécuteur s'exécute automatiquement. Il contient le code de production qui devrait finalement produire le résultat. Ses arguments **resolve** et **reject** sont des fonctions de rappels fournis par JavaScript lui-même. Notre code se trouve uniquement à l'intérieur de l'exécuteur. Lorsque l'exécuteur obtient le résultat, que ce soit tôt ou tard, peu importe, il doit appeler l'un de ces rappels :

- `resolve(value)` - si le travail est terminé avec succès, avec la valeur du résultat.
- `reject(error)` - si une erreur s'est produite, `error` est l'objet d'erreur.

JavaScript ES6: Promises

Voici un exemple d'un constructeur de promesses et d'une simple fonction d'exécution avec un "code de production" qui prend du temps.

```
let promise = new Promise(function(resolve, reject) {  
  // la fonction est exécutée automatiquement lorsque la promesse est construite  
  // après 1 seconde, signaler que le travail est terminé avec le résultat "done".  
  setTimeout(() => resolve("done"), 1000);  
});
```

Voici maintenant un exemple où l'exécuteur rejette la promesse avec une erreur

```
let promise = new Promise(function(resolve, reject) {  
  // après 1 seconde, signale que le travail est terminé avec une erreur  
  setTimeout(() => reject(new Error("Whoops!")), 1000);  
});
```

JavaScript ES6: Promises

Un objet Promise sert de lien entre l'exécuteur et les fonctions consommatrices, qui recevront le résultat ou l'erreur. Les fonctions consommatrices peuvent être enregistrées à l'aide des méthodes **.then()** et **.catch()**

Syntaxe de la méthode .then():

```
promise.then(  
  function(result) { /* handle a successful result */ },  
  function(error) { /* handle an error */ }  
);
```

Le premier argument de .then est une fonction qui s'exécute lorsque la promesse est résolue et reçoit le résultat. Le deuxième argument de .then est une fonction qui s'exécute lorsque la promesse est rejetée et qui reçoit l'erreur.

Par exemple, voici une réaction à une promesse résolue avec succès :

```
let promise = new Promise(function(resolve, reject) {  
  setTimeout(() => resolve("done!"), 1000);  
});  
  
promise.then(  
  result => alert(result), // shows "done!" after 1 second  
  error => alert(error) // doesn't run  
);
```

JavaScript ES6: Promises

Et en cas de rejet, la seconde fonction est appelée:

```
let promise = new Promise(function(resolve, reject) {
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});

promise.then(
  result => alert(result), // doesn't run
  error => alert(error) // shows "Error: Whoops!" after 1 second
);
```

Si on est intéressé seulement par les erreurs, nous pouvons utiliser `.catch(errorHandlingFunction)` :

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});

// .catch(f) is the same as promise.then(null, f)
promise.catch(alert); // shows "Error: Whoops!" after 1 second
```

JavaScript ES6: Async/await

Il existe une syntaxe spéciale pour travailler avec les promesses de manière plus confortable, appelée "async/await"

Syntaxe d'async:

```
async function functionName() {  
    return ...;  
}
```

Le mot "async" devant une fonction signifie une chose simple : une fonction renvoie toujours une promesse. Les autres valeurs sont automatiquement enveloppées dans une promesse résolue.

Par exemple, cette fonction renvoie une promesse résolue avec le résultat 1:

```
async function f() {  
    return 1;  
}  
  
f().then(alert); // 1
```

JavaScript ES6: Async/await

Syntaxe d'await:

```
// works only inside async functions  
let value = await promise;
```

Le mot-clé **await** fait attendre jusqu'à ce que la promesse s'établisse et renvoie son résultat. **Await est utilisée uniquement au sein des fonctions Async!**

Voici un exemple avec une promesse qui se résout en 1 seconde:

```
async function f() {  
  let promise = new Promise((resolve, reject) => {  
    setTimeout(() => resolve("done!"), 1000)  
  });  
  
  let result = await promise; // wait until the promise resolves (*)  
  alert(result); // "done!"  
}  
f();
```

L'exécution de la fonction "s'interrompt" à la ligne (*) et reprend lorsque la promesse se réalise, le résultat devenant son résultat. Ainsi, le code ci-dessus affiche "done !" en une seconde

JavaScript ES6: ES Modules + Import / Export syntax

ES6 nous a donné la possibilité de partager facilement du code entre nos propres fichiers JavaScript et des bibliothèques tierces à l'aide de modules ES. De plus, lorsque nous utilisons des outils tels que Webpack, nous pouvons importer des ressources telles que des images et des fichiers CSS et les utiliser comme valeurs dynamiques dans notre code.

```
/* We're bringing into our file a library (React), a png image, and CSS styles */
import React from "react";
import logo from "../img/site-logo.png";
import "../styles/app.css";

function App() {
  return (
    <div>
      Welcome!
      <img src={logo} alt="Site logo" />
    </div>
  );
}

export default App;
```

Un module n'est qu'un fichier. Un script est un module. C'est aussi simple que cela. Les modules peuvent se charger les uns les autres et utiliser les directives spéciales **export** et **import** pour échanger des fonctionnalités, appeler des fonctions d'un module à partir d'un autre :

- le mot-clé **export** désigne les variables et les fonctions qui doivent être accessibles depuis l'extérieur du module en cours.
- **import** permet d'importer des fonctionnalités d'autres modules.

JavaScript ES6: ES Modules + Import / Export syntax

L'idée derrière les modules ES est de pouvoir diviser notre code JavaScript en différents fichiers, afin de le rendre modulaire ou réutilisable dans notre application. En ce qui concerne le code JavaScript, nous pouvons importer et exporter des variables et des fonctions. Il existe deux façons d'importer et d'exporter : les importations/exportations nommées et les importations/exportations par défaut. Il ne peut y avoir qu'une seule chose que nous importons ou exportons par défaut par fichier et nous pouvons nommer autant de choses import/export que nous le souhaitons. Par exemple :

```
// constants.js
export const name = "ALAMI";
export const age = 28;
export default function getName() {
  return name;
}

// app.js
// Notez que les exportations nommées sont importées entre accolades
import getName, { name, age } from "../constants.js";
console.log(name, age, getName());
```

Les modules fonctionnent uniquement via HTTP(s), pas localement. Si vous essayez d'ouvrir une page web localement, via le protocole file://, vous constaterez que les directives d'importation et d'exportation ne fonctionnent pas. Utilisez un serveur web local, tel que static-server, ou utilisez la fonction "live server" de votre éditeur, telle que VS Code Live Server Extension pour tester les modules.

JavaScript ES6: ES Modules + Import / Export syntax

Nous pouvons également écrire toutes les exportations à la fin du fichier plutôt qu'à côté de chaque variable ou fonction :

```
// constants.js
const name = "ALAMI";

const age = 28;

function getName() {
  return name;
}

export { name, age };
export default getName;

// app.js
import getName, { name, age } from "../constants.js";

console.log(name, age, getName());
```

JavaScript ES6: ES Modules + Import / Export syntax

Vous pouvez également donner un alias ou un nouveau nom à ce que vous importez en utilisant le mot-clé `as` pour les importations nommées. L'avantage des exportations par défaut est qu'elles peuvent être nommées comme vous le souhaitez.

```
// constants.js
const name = "ALAMI";

const age = 28;

function getName() {
  return name;
}

export { name, age };
export default getName;

// app.js
import getName, { name as myName, age as myAge } from "../constants.js";

console.log(myName, myAge, getName());
```

JavaScript ES6: Closure in JavaScript

La fermeture est la caractéristique de JavaScript qui permet à une fonction interne d'avoir accès à son champ d'application actuel ainsi qu'à son champ d'application parent lexical. Il s'agit d'un concept très fondamental, basique mais important de JavaScript. La fermeture a la propriété d'encapsuler, d'organiser le code et de mémoriser la portée de son parent lexical. Dans react.js, la vie privée peut être réalisée à l'aide de fermetures, en créant une logique encapsulée à l'intérieur des composants.

```
function start() {  
  var content = "ENIAD"; // content est une variable locale créée par start()  
  function company() {  
    // company() est la fonction interne, qui forme la fermeture  
    console.log(content); // utilisation d'une variable déclarée dans la fonction mère  
  }  
  company();  
}  
start();
```

JavaScript ES6: Classes

JavaScript a introduit la syntaxe de classe dans ES6 pour fournir une approche POO plus familière. Dans ce paradigme, vous définissez les classes à l'aide du mot-clé "class".

Exemple:

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  area() {
    return this.width * this.height;
  }

  perimeter() {
    return 2 * (this.width + this.height);
  }
}

const rect = new Rectangle(5, 10);
console.log(rect.area());    // Output: 50
console.log(rect.perimeter()); // Output: 30
```

Le constructeur est défini à l'aide de la méthode prédéfinie **constructor**. Une classe peut hériter d'une autre à l'aide du mot clé **extends**

Architecture et fonctionnement de React

React: introduction

React (Ou ReactJS, React.js) est une bibliothèque JavaScript développée par Facebook, utilisée pour créer des composants d'affichage réutilisables.

Pour créer une application avec React, on va donc créer des composants d'affichage (en général des classes ou des fonctions) qui seront ensuite assemblés afin de former l'application finale. L'intérêt de React, outre le fait de créer des composants réutilisables, et de permettre aussi une mise à jour rapide de la page HTML.

En effet, lors de l'utilisation de bibliothèques telles que jQuery (qui manipulent les éléments HTML de la page, c'est-à-dire le DOM, directement), les performances sont bien moindres car la manipulation du DOM directement en JavaScript est lente lors de l'exécution. React ne manipule pas le DOM directement mais une copie interne de celui-ci (les objets ou éléments React, appelés le DOM virtuel), et produit les modifications sur l'affichage uniquement lorsque cela s'avère nécessaire.

Enfin, React n'est pas seulement une bibliothèque permettant d'effectuer des affichages sur le Web, elle peut également servir à produire des applications natives iPhone, Android et desktop, en utilisant une variante nommée React Native.

L'écosystème React: npm

npm, acronyme de **N**ode **P**ackage **M**anager, est le gestionnaire de paquets par défaut pour l'écosystème Node.js. Il permet aux développeurs JavaScript d'installer, de partager et de gérer les dépendances de leurs projets Node.js de manière efficace. C'est l'équivalent de composer pour le PHP.

Ci-dessous les fonctionnalités les plus importantes d'npm :

- **Installation de paquets** : npm permet d'installer des paquets JavaScript (bibliothèques, frameworks, outils, etc.) à l'aide de la commande `npm install`. Ces paquets sont généralement hébergés sur le registre npm public, mais peuvent également être installés à partir de sources privées ou locales.
- **Gestion des dépendances** : Les développeurs peuvent spécifier les dépendances de leurs projets dans un fichier `package.json`, qui décrit les métadonnées du projet ainsi que les dépendances requises. npm gère alors ces dépendances en installant automatiquement les paquets nécessaires et en les mettant à jour si nécessaire.
- **Scripts personnalisés** : npm permet également de définir des scripts personnalisés dans le fichier `package.json`, ce qui permet d'exécuter des tâches telles que la compilation de code, les tests, le déploiement, etc., en utilisant des commandes simples telles que `npm run script-name`.
- **Communauté et écosystème** : npm héberge l'un des plus vastes écosystèmes de paquets open source au monde, offrant des milliers de modules prêts à l'emploi pour une variété de tâches de développement. Cela favorise la réutilisabilité du code et accélère le processus de développement.

npm est un outil essentiel pour les développeurs JavaScript travaillant avec React

L'écosystème React: Babel

Babel est un compilateur JavaScript populaire à code source ouvert qui permet aux développeurs d'écrire du code JavaScript moderne en utilisant les dernières fonctionnalités du langage, puis de convertir ce code dans un format qui peut être exécuté sur des navigateurs ou des environnements plus anciens. Il est couramment utilisé dans le développement web pour s'assurer que le code écrit dans la dernière version de JavaScript est compatible avec un plus grand nombre de navigateurs.

Babel fonctionne en prenant le code JavaScript écrit à l'aide de nouvelles syntaxes et API de langage qui ne sont pas encore prises en charge par tous les navigateurs, et en le transformant en une version de JavaScript qui est largement prise en charge. Ci-dessous un exemple de transformation faite par Babel.

```
// ES6 code
const greet = (name) => {
  console.log(`Hello, ${name}!`);
}

greet('ALAMI');

// Transpiled ES5 code
'use strict';

var greet = function greet(name) {
  console.log('Hello, ' + name + '!');
};

greet('ALAMI');
```

L'écosystème React: Bundlers

Le **bundler** condense tout le code de votre application pour n'avoir qu'un seul fichier JavaScript à importer dans votre page HTML. L'intérêt majeur c'est que vous n'êtes plus contraints d'avoir un fichier volumineux quand vous codez avec JavaScript, où il est impossible de retrouver votre fonction. Vous pouvez faire des modules (fichiers isolés), y faire référence dans un autre fichier et tout de même être capable de les utiliser dans le navigateur sans ajouter une balise `<script>` à chaque fois que vous ajoutez un fichier. Les deux bundlers majeurs sur le marché actuellement sont **Webpack** et **Vite**

